Contents

DSA Solutions Guide - Part 5: Binary Search, Heaps, Tries & More **2**
Complete JavaScript Solutions with Detailed Explanations . . . 2

BINARY SEARCH **2**
Understanding Binary Search . . . 2
Binary Search Template . . . 2

1. Search Insert Position **3**
Problem . . . 3

2. Find First and Last Position **3**
Problem . . . 3

3. Search in Rotated Sorted Array **4**
Problem . . . 4

4. Find Minimum in Rotated Sorted Array **5**

5. Find Peak Element **6**
Problem . . . 6

6. Search a 2D Matrix **6**
Problem . . . 6

7. Koko Eating Bananas **7**
Problem . . . 7

8. Capacity To Ship Packages **7**
Problem . . . 7

9. Split Array Largest Sum **8**
Problem . . . 8

HEAPS / PRIORITY QUEUES **9**
Heap Implementation . . . 9

10. Kth Largest Element in Array **10**
Problem . . . 10

11. Top K Frequent Elements **11**

12. Find Median from Data Stream **13**

13. Task Scheduler **14**
Problem . . . 14
Explanation . . . 15

TRIES **15**
Trie Implementation . . . 15

14. Design Add and Search Words **16**
Problem . . . 16

15. Word Search II **17**
Problem . . . 17

BIT MANIPULATION **18**
Common Operations . . . 18

16. Single Number **19**
Problem . . . 19

17. Single Number II **19**
Problem . . . 19

18. Missing Number **20**
Problem . . . 20

19. Sum of Two Integers (Without + or -) **20**

20. Reverse Bits **21**

GREEDY ALGORITHMS **21**

21. Jump Game **21**
Problem . . . 21

22. Jump Game II **21**
Problem . . . 21

23. Gas Station **22**
Problem . . . 22

24. Partition Labels **22**
Problem . . . 22

INTERVALS **23**

25. Meeting Rooms II **23**
Problem . . . 23

Summary: Key Patterns **24**
Binary Search Patterns . . . 24
Heap Patterns . . . 24
Bit Manipulation . . . 24
Greedy Patterns . . . 25

Time Complexity Cheat Sheet **25**

DSA Solutions Guide - Part 5: Binary Search, Heaps, Tries & More

Complete JavaScript Solutions with Detailed Explanations

BINARY SEARCH

Understanding Binary Search

Binary search is an efficient algorithm for finding an item in a **sorted** list.

Key Points: - Requires sorted input - Eliminates half the search space each iteration - Time: O(log n), Space: O(1)

Binary Search Template

```
// Standard Template
function binarySearch(arr, target) {
    let left = 0;
```

```
    let right = arr.length - 1;

    while (left <= right) {
        const mid = Math.floor((left + right) / 2);
        // Or: mid = left + Math.floor((right - left) / 2); // Prevents overflow

        if (arr[mid] === target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;  // Not found
}
```

---

1. Search Insert Position

Problem

Find index where target should be inserted in sorted array.

```
function searchInsert(nums, target) {
    let left = 0;
    let right = nums.length;

    while (left < right) {
        const mid = Math.floor((left + right) / 2);

        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }

    return left;
}

// Time: O(log n), Space: O(1)
```

---

2. Find First and Last Position

Problem

Find starting and ending position of target in sorted array.

```
function searchRange(nums, target) {
    return [findFirst(nums, target), findLast(nums, target)];
}

function findFirst(nums, target) {
```

```
    let left = 0;
    let right = nums.length - 1;
    let result = -1;

    while (left <= right) {
        const mid = Math.floor((left + right) / 2);

        if (nums[mid] === target) {
            result = mid;
            right = mid - 1;  // Keep searching left
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return result;
}

function findLast(nums, target) {
    let left = 0;
    let right = nums.length - 1;
    let result = -1;

    while (left <= right) {
        const mid = Math.floor((left + right) / 2);

        if (nums[mid] === target) {
            result = mid;
            left = mid + 1;  // Keep searching right
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return result;
}

// Time: O(log n), Space: O(1)
```

---

3. Search in Rotated Sorted Array

Problem

Search in a rotated sorted array (e.g., [4,5,6,7,0,1,2]).

```
function search(nums, target) {
    let left = 0;
    let right = nums.length - 1;

    while (left <= right) {
        const mid = Math.floor((left + right) / 2);
```

```
        if (nums[mid] === target) return mid;

        // Determine which half is sorted
        if (nums[left] <= nums[mid]) {
            // Left half is sorted
            if (target >= nums[left] && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            // Right half is sorted
            if (target > nums[mid] && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }

    return -1;
}

// Time: O(log n), Space: O(1)
```

---

4. Find Minimum in Rotated Sorted Array

```
function findMin(nums) {
    let left = 0;
    let right = nums.length - 1;

    while (left < right) {
        const mid = Math.floor((left + right) / 2);

        if (nums[mid] > nums[right]) {
            // Minimum is in right half
            left = mid + 1;
        } else {
            // Minimum is in left half (including mid)
            right = mid;
        }
    }

    return nums[left];
}

// Time: O(log n), Space: O(1)
```

---

5. Find Peak Element

Problem

Find any peak element (element greater than neighbors).

```
function findPeakElement(nums) {
    let left = 0;
    let right = nums.length - 1;

    while (left < right) {
        const mid = Math.floor((left + right) / 2);

        if (nums[mid] > nums[mid + 1]) {
            // Peak is in left half (including mid)
            right = mid;
        } else {
            // Peak is in right half
            left = mid + 1;
        }
    }

    return left;
}

// Time: O(log n), Space: O(1)
```

---

6. Search a 2D Matrix

Problem

Search in a matrix where each row is sorted and first element of each row > last element of previous row.

```
function searchMatrix(matrix, target) {
    if (matrix.length === 0) return false;

    const m = matrix.length;
    const n = matrix[0].length;

    // Treat as 1D sorted array
    let left = 0;
    let right = m * n - 1;

    while (left <= right) {
        const mid = Math.floor((left + right) / 2);
        const midVal = matrix[Math.floor(mid / n)][mid % n];

        if (midVal === target) {
            return true;
        } else if (midVal < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
```

```
    return false;
}

// Time: O(log(m*n)), Space: O(1)
```

---

7. Koko Eating Bananas

Problem

Koko can eat k bananas per hour. Find minimum k to eat all piles within h hours.

```
function minEatingSpeed(piles, h) {
    let left = 1;
    let right = Math.max(...piles);

    while (left < right) {
        const mid = Math.floor((left + right) / 2);

        if (canFinish(piles, mid, h)) {
            right = mid;  // Try smaller speed
        } else {
            left = mid + 1;
        }
    }

    return left;
}

function canFinish(piles, speed, h) {
    let hours = 0;
    for (const pile of piles) {
        hours += Math.ceil(pile / speed);
    }
    return hours <= h;
}

// Time: O(n * log(max(piles))), Space: O(1)
```

---

8. Capacity To Ship Packages

Problem

Find minimum ship capacity to ship all packages within D days.

```
function shipWithinDays(weights, days) {
    let left = Math.max(...weights);  // Min capacity: heaviest package
    let right = weights.reduce((a, b) => a + b, 0);  // Max: all in one day

    while (left < right) {
        const mid = Math.floor((left + right) / 2);

        if (canShip(weights, mid, days)) {
            right = mid;
        } else {
```

```
            left = mid + 1;
        }
    }

    return left;
}

function canShip(weights, capacity, days) {
    let daysNeeded = 1;
    let currentLoad = 0;

    for (const weight of weights) {
        if (currentLoad + weight > capacity) {
            daysNeeded++;
            currentLoad = 0;
        }
        currentLoad += weight;
    }

    return daysNeeded <= days;
}

// Time: O(n * log(sum)), Space: O(1)
```

---

9. Split Array Largest Sum

Problem

Split array into m subarrays to minimize the largest sum.

```
function splitArray(nums, m) {
    let left = Math.max(...nums);
    let right = nums.reduce((a, b) => a + b, 0);

    while (left < right) {
        const mid = Math.floor((left + right) / 2);

        if (canSplit(nums, m, mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    return left;
}

function canSplit(nums, m, maxSum) {
    let count = 1;
    let currentSum = 0;

    for (const num of nums) {
        if (currentSum + num > maxSum) {
            count++;
            currentSum = 0;
```

```
        }
        currentSum += num;
    }

    return count <= m;
}

// Time: O(n * log(sum)), Space: O(1)
```

HEAPS / PRIORITY QUEUES

Heap Implementation

```
class MinHeap {
    constructor() {
        this.heap = [];
    }

    size() {
        return this.heap.length;
    }

    peek() {
        return this.heap[0];
    }

    push(val) {
        this.heap.push(val);
        this._bubbleUp(this.heap.length - 1);
    }

    pop() {
        if (this.heap.length === 0) return undefined;

        const min = this.heap[0];
        const last = this.heap.pop();

        if (this.heap.length > 0) {
            this.heap[0] = last;
            this._bubbleDown(0);
        }

        return min;
    }

    _bubbleUp(index) {
        while (index > 0) {
            const parentIdx = Math.floor((index - 1) / 2);
            if (this.heap[parentIdx] <= this.heap[index]) break;

            [this.heap[parentIdx], this.heap[index]] = [this.heap[index], this.heap[parentIdx]];
            index = parentIdx;
        }
    }
```

```
    _bubbleDown(index) {
        const length = this.heap.length;

        while (true) {
            let smallest = index;
            const leftChild = 2 * index + 1;
            const rightChild = 2 * index + 2;

            if (leftChild < length && this.heap[leftChild] < this.heap[smallest]) {
                smallest = leftChild;
            }
            if (rightChild < length && this.heap[rightChild] < this.heap[smallest]) {
                smallest = rightChild;
            }

            if (smallest === index) break;

            [this.heap[smallest], this.heap[index]] = [this.heap[index], this.heap[smallest]];
            index = smallest;
        }
    }
}

// For MaxHeap: negate values or flip comparisons
```

---

10. Kth Largest Element in Array

Problem

Find the kth largest element.

```
// Using Min Heap of size k
function findKthLargest(nums, k) {
    const minHeap = new MinHeap();

    for (const num of nums) {
        minHeap.push(num);
        if (minHeap.size() > k) {
            minHeap.pop();
        }
    }

    return minHeap.peek();
}

// QuickSelect - Average O(n)
function findKthLargestQuickSelect(nums, k) {
    const targetIdx = nums.length - k;
    return quickSelect(nums, 0, nums.length - 1, targetIdx);
}

function quickSelect(nums, left, right, targetIdx) {
    if (left === right) return nums[left];

    const pivotIdx = partition(nums, left, right);
```

```
    if (pivotIdx === targetIdx) {
        return nums[pivotIdx];
    } else if (pivotIdx < targetIdx) {
        return quickSelect(nums, pivotIdx + 1, right, targetIdx);
    } else {
        return quickSelect(nums, left, pivotIdx - 1, targetIdx);
    }
}

function partition(nums, left, right) {
    const pivot = nums[right];
    let i = left;

    for (let j = left; j < right; j++) {
        if (nums[j] < pivot) {
            [nums[i], nums[j]] = [nums[j], nums[i]];
            i++;
        }
    }

    [nums[i], nums[right]] = [nums[right], nums[i]];
    return i;
}

// Heap: Time O(n log k), Space O(k)
// QuickSelect: Time O(n) avg, O(n^2) worst, Space O(1)
```

---

11. Top K Frequent Elements

```
function topKFrequent(nums, k) {
    // Count frequencies
    const freqMap = new Map();
    for (const num of nums) {
        freqMap.set(num, (freqMap.get(num) || 0) + 1);
    }

    // Bucket sort approach - O(n)
    const buckets = new Array(nums.length + 1).fill(null).map(() => []);

    for (const [num, freq] of freqMap) {
        buckets[freq].push(num);
    }

    // Collect top k from highest frequency buckets
    const result = [];
    for (let i = buckets.length - 1; i >= 0 && result.length < k; i--) {
        result.push(...buckets[i]);
    }

    return result.slice(0, k);
}

// Using Min Heap
```

```javascript
function topKFrequentHeap(nums, k) {
    const freqMap = new Map();
    for (const num of nums) {
        freqMap.set(num, (freqMap.get(num) || 0) + 1);
    }

    // Min heap by frequency
    const minHeap = [];

    const push = (item) => {
        minHeap.push(item);
        let i = minHeap.length - 1;
        while (i > 0) {
            const parent = Math.floor((i - 1) / 2);
            if (minHeap[parent][1] <= minHeap[i][1]) break;
            [minHeap[parent], minHeap[i]] = [minHeap[i], minHeap[parent]];
            i = parent;
        }
    };

    const pop = () => {
        const min = minHeap[0];
        const last = minHeap.pop();
        if (minHeap.length > 0) {
            minHeap[0] = last;
            let i = 0;
            while (true) {
                let smallest = i;
                const left = 2 * i + 1;
                const right = 2 * i + 2;
                if (left < minHeap.length && minHeap[left][1] < minHeap[smallest][1]) smallest = left;
                if (right < minHeap.length && minHeap[right][1] < minHeap[smallest][1]) smallest = right;
                if (smallest === i) break;
                [minHeap[smallest], minHeap[i]] = [minHeap[i], minHeap[smallest]];
                i = smallest;
            }
        }
        return min;
    };

    for (const [num, freq] of freqMap) {
        push([num, freq]);
        if (minHeap.length > k) pop();
    }

    return minHeap.map(item => item[0]);
}

// Bucket: Time O(n), Space O(n)
// Heap: Time O(n log k), Space O(n)
```

---

12. Find Median from Data Stream

```
class MedianFinder {
    constructor() {
        this.maxHeap = [];  // Lower half (store negated values)
        this.minHeap = [];  // Upper half
    }

    addNum(num) {
        // Add to max heap (lower half)
        this._pushMax(num);

        // Balance: move max of lower half to upper half
        this._pushMin(-this._popMax());

        // If upper half has more elements, move one back
        if (this.minHeap.length > this.maxHeap.length) {
            this._pushMax(-this._popMin());
        }
    }

    findMedian() {
        if (this.maxHeap.length > this.minHeap.length) {
            return -this.maxHeap[0];
        }
        return (-this.maxHeap[0] + this.minHeap[0]) / 2;
    }

    _pushMax(val) {
        this.maxHeap.push(-val);
        this._bubbleUpMax(this.maxHeap.length - 1);
    }

    _popMax() {
        const max = this.maxHeap[0];
        const last = this.maxHeap.pop();
        if (this.maxHeap.length > 0) {
            this.maxHeap[0] = last;
            this._bubbleDownMax(0);
        }
        return max;
    }

    _pushMin(val) {
        this.minHeap.push(val);
        this._bubbleUpMin(this.minHeap.length - 1);
    }

    _popMin() {
        const min = this.minHeap[0];
        const last = this.minHeap.pop();
        if (this.minHeap.length > 0) {
            this.minHeap[0] = last;
            this._bubbleDownMin(0);
        }
        return min;
```

```
    }

    _bubbleUpMax(i) {
        while (i > 0) {
            const p = Math.floor((i - 1) / 2);
            if (this.maxHeap[p] <= this.maxHeap[i]) break;
            [this.maxHeap[p], this.maxHeap[i]] = [this.maxHeap[i], this.maxHeap[p]];
            i = p;
        }
    }

    _bubbleDownMax(i) {
        while (true) {
            let smallest = i;
            const l = 2 * i + 1, r = 2 * i + 2;
            if (l < this.maxHeap.length && this.maxHeap[l] < this.maxHeap[smallest]) smallest = l;
            if (r < this.maxHeap.length && this.maxHeap[r] < this.maxHeap[smallest]) smallest = r;
            if (smallest === i) break;
            [this.maxHeap[smallest], this.maxHeap[i]] = [this.maxHeap[i], this.maxHeap[smallest]];
            i = smallest;
        }
    }

    _bubbleUpMin(i) {
        while (i > 0) {
            const p = Math.floor((i - 1) / 2);
            if (this.minHeap[p] <= this.minHeap[i]) break;
            [this.minHeap[p], this.minHeap[i]] = [this.minHeap[i], this.minHeap[p]];
            i = p;
        }
    }

    _bubbleDownMin(i) {
        while (true) {
            let smallest = i;
            const l = 2 * i + 1, r = 2 * i + 2;
            if (l < this.minHeap.length && this.minHeap[l] < this.minHeap[smallest]) smallest = l;
            if (r < this.minHeap.length && this.minHeap[r] < this.minHeap[smallest]) smallest = r;
            if (smallest === i) break;
            [this.minHeap[smallest], this.minHeap[i]] = [this.minHeap[i], this.minHeap[smallest]];
            i = smallest;
        }
    }
}

// addNum: O(log n), findMedian: O(1), Space: O(n)
```

---

13. Task Scheduler

Problem

Schedule tasks with cooldown period n between same tasks.

```
function leastInterval(tasks, n) {
    // Count task frequencies
```

```
    const freq = new Array(26).fill(0);
    for (const task of tasks) {
        freq[task.charCodeAt(0) - 65]++;
    }

    // Find max frequency
    const maxFreq = Math.max(...freq);

    // Count how many tasks have max frequency
    const maxCount = freq.filter(f => f === maxFreq).length;

    // Formula: (maxFreq - 1) * (n + 1) + maxCount
    // This creates slots with gaps
    const intervals = (maxFreq - 1) * (n + 1) + maxCount;

    // Return max of formula and total tasks (no idle if tasks fill all slots)
    return Math.max(intervals, tasks.length);
}

// Time: O(n), Space: O(1)
```

Explanation

```
tasks = [A,A,A,B,B,B], n = 2

maxFreq = 3 (A and B both appear 3 times)
maxCount = 2

Layout: A _ _ A _ _ A
        A B _ A B _ A B

intervals = (3-1) * (2+1) + 2 = 8
But we have 6 tasks, so answer = max(8, 6) = 8
```

TRIES

Trie Implementation

```
class TrieNode {
    constructor() {
        this.children = {};
        this.isEnd = false;
    }
}

class Trie {
    constructor() {
        this.root = new TrieNode();
    }

    insert(word) {
        let node = this.root;
        for (const char of word) {
            if (!node.children[char]) {
                node.children[char] = new TrieNode();
```

```
            }
            node = node.children[char];
        }
        node.isEnd = true;
    }

    search(word) {
        const node = this._traverse(word);
        return node !== null && node.isEnd;
    }

    startsWith(prefix) {
        return this._traverse(prefix) !== null;
    }

    _traverse(str) {
        let node = this.root;
        for (const char of str) {
            if (!node.children[char]) {
                return null;
            }
            node = node.children[char];
        }
        return node;
    }
}

// Insert/Search/StartsWith: O(m) where m = word length
// Space: O(n * m) where n = number of words
```

---

14. Design Add and Search Words

Problem

Design a data structure that supports adding words and searching with '.' as wildcard.

```
class WordDictionary {
    constructor() {
        this.root = {};
    }

    addWord(word) {
        let node = this.root;
        for (const char of word) {
            if (!node[char]) {
                node[char] = {};
            }
            node = node[char];
        }
        node.isEnd = true;
    }

    search(word) {
        return this._searchFrom(word, 0, this.root);
    }
```

```
    _searchFrom(word, index, node) {
        if (index === word.length) {
            return node.isEnd === true;
        }

        const char = word[index];

        if (char === '.') {
            // Try all possible children
            for (const key in node) {
                if (key !== 'isEnd' && this._searchFrom(word, index + 1, node[key])) {
                    return true;
                }
            }
            return false;
        } else {
            if (!node[char]) return false;
            return this._searchFrom(word, index + 1, node[char]);
        }
    }
}

// addWord: O(m), search: O(26^m) worst case with all dots
```

---

15. Word Search II

Problem

Find all dictionary words that exist in a 2D board.

```
function findWords(board, words) {
    // Build Trie from words
    const root = {};
    for (const word of words) {
        let node = root;
        for (const char of word) {
            if (!node[char]) node[char] = {};
            node = node[char];
        }
        node.word = word;  // Store word at end
    }

    const result = [];
    const rows = board.length;
    const cols = board[0].length;

    function dfs(r, c, node) {
        const char = board[r][c];

        if (!node[char]) return;

        const nextNode = node[char];

        // Found a word
```

```
        if (nextNode.word) {
            result.push(nextNode.word);
            nextNode.word = null;  // Avoid duplicates
        }

        // Mark visited
        board[r][c] = '#';

        // Explore neighbors
        const dirs = [[0, 1], [0, -1], [1, 0], [-1, 0]];
        for (const [dr, dc] of dirs) {
            const nr = r + dr;
            const nc = c + dc;
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && board[nr][nc] !== '#') {
                dfs(nr, nc, nextNode);
            }
        }

        // Restore
        board[r][c] = char;
    }

    // Start DFS from each cell
    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            dfs(r, c, root);
        }
    }

    return result;
}

// Time: O(m * n * 4^L) where L = max word length
// Space: O(sum of word lengths)
```

BIT MANIPULATION

Common Operations

```
// Check if bit at position i is set
function getBit(num, i) {
    return (num >> i) & 1;
}

// Set bit at position i
function setBit(num, i) {
    return num | (1 << i);
}

// Clear bit at position i
function clearBit(num, i) {
    return num & ~(1 << i);
}

// Toggle bit at position i
```

```
function toggleBit(num, i) {
    return num ^ (1 << i);
}

// Check if power of 2
function isPowerOfTwo(n) {
    return n > 0 && (n & (n - 1)) === 0;
}

// Count set bits (Brian Kernighan)
function countBits(n) {
    let count = 0;
    while (n) {
        n &= (n - 1);  // Clear lowest set bit
        count++;
    }
    return count;
}
```

---

16. Single Number

Problem

Find the element that appears only once (others appear twice).

```
function singleNumber(nums) {
    let result = 0;
    for (const num of nums) {
        result ^= num;  // XOR: a ^ a = 0, a ^ 0 = a
    }
    return result;
}

// Time: O(n), Space: O(1)
```

---

17. Single Number II

Problem

Find element appearing once when others appear three times.

```
function singleNumber2(nums) {
    let ones = 0;   // Bits that have appeared once
    let twos = 0;   // Bits that have appeared twice

    for (const num of nums) {
        // Add to ones if not in twos
        ones = (ones ^ num) & ~twos;
        // Add to twos if not in ones
        twos = (twos ^ num) & ~ones;
    }

    return ones;
}
```

```javascript
// Alternative: Count bits
function singleNumber2Count(nums) {
    let result = 0;

    for (let i = 0; i < 32; i++) {
        let sum = 0;
        for (const num of nums) {
            sum += (num >> i) & 1;
        }
        result |= (sum % 3) << i;
    }

    // Handle negative numbers in JavaScript
    return result | 0;
}

// Time: O(n), Space: O(1)
```

---

18. Missing Number

Problem

Find missing number in range [0, n].

```javascript
// XOR approach
function missingNumber(nums) {
    let xor = nums.length;

    for (let i = 0; i < nums.length; i++) {
        xor ^= i ^ nums[i];
    }

    return xor;
}

// Sum approach
function missingNumberSum(nums) {
    const n = nums.length;
    const expectedSum = (n * (n + 1)) / 2;
    const actualSum = nums.reduce((a, b) => a + b, 0);
    return expectedSum - actualSum;
}

// Time: O(n), Space: O(1)
```

---

19. Sum of Two Integers (Without + or -)

```javascript
function getSum(a, b) {
    while (b !== 0) {
        const carry = (a & b) << 1;  // Carry bits
        a = a ^ b;                   // Sum without carry
        b = carry;
```

```
    }
    return a;
}

// Time: O(1), Space: O(1)
```

---

20. Reverse Bits

```
function reverseBits(n) {
    let result = 0;

    for (let i = 0; i < 32; i++) {
        result = (result << 1) | (n & 1);
        n >>>= 1;  // Unsigned right shift
    }

    return result >>> 0;  // Convert to unsigned
}

// Time: O(1), Space: O(1)
```

GREEDY ALGORITHMS

---

21. Jump Game

Problem

Determine if you can reach the last index.

```
function canJump(nums) {
    let maxReach = 0;

    for (let i = 0; i < nums.length; i++) {
        if (i > maxReach) return false;
        maxReach = Math.max(maxReach, i + nums[i]);
    }

    return true;
}

// Time: O(n), Space: O(1)
```

---

22. Jump Game II

Problem

Minimum number of jumps to reach the end.

```
function jump(nums) {
    let jumps = 0;
    let currentEnd = 0;
    let farthest = 0;

    for (let i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);

        if (i === currentEnd) {
            jumps++;
            currentEnd = farthest;
        }
    }

    return jumps;
}

// Time: O(n), Space: O(1)
```

---

23. Gas Station

Problem

Find starting gas station to complete circular route.

```
function canCompleteCircuit(gas, cost) {
    let totalTank = 0;
    let currentTank = 0;
    let startStation = 0;

    for (let i = 0; i < gas.length; i++) {
        const net = gas[i] - cost[i];
        totalTank += net;
        currentTank += net;

        if (currentTank < 0) {
            // Can't reach station i+1 from startStation
            startStation = i + 1;
            currentTank = 0;
        }
    }

    return totalTank >= 0 ? startStation : -1;
}

// Time: O(n), Space: O(1)
```

---

24. Partition Labels

Problem

Partition string so each letter appears in at most one part.

```
function partitionLabels(s) {
    // Find last occurrence of each character
    const lastIndex = {};
    for (let i = 0; i < s.length; i++) {
        lastIndex[s[i]] = i;
    }

    const result = [];
    let start = 0;
    let end = 0;

    for (let i = 0; i < s.length; i++) {
        end = Math.max(end, lastIndex[s[i]]);

        if (i === end) {
            result.push(end - start + 1);
            start = i + 1;
        }
    }

    return result;
}

// Time: O(n), Space: O(1)
```

INTERVALS

---

25. Meeting Rooms II

Problem

Find minimum number of meeting rooms required.

```
function minMeetingRooms(intervals) {
    if (intervals.length === 0) return 0;

    const starts = intervals.map(i => i[0]).sort((a, b) => a - b);
    const ends = intervals.map(i => i[1]).sort((a, b) => a - b);

    let rooms = 0;
    let endPtr = 0;

    for (let i = 0; i < starts.length; i++) {
        if (starts[i] < ends[endPtr]) {
            // Need a new room
            rooms++;
        } else {
            // Reuse a room
            endPtr++;
        }
    }

    return rooms;
```

```
}

// Using Min Heap
function minMeetingRoomsHeap(intervals) {
    if (intervals.length === 0) return 0;

    intervals.sort((a, b) => a[0] - b[0]);

    const heap = new MinHeap();
    heap.push(intervals[0][1]);  // End time of first meeting

    for (let i = 1; i < intervals.length; i++) {
        // If earliest ending meeting is done before this starts
        if (heap.peek() <= intervals[i][0]) {
            heap.pop();  // Reuse room
        }
        heap.push(intervals[i][1]);
    }

    return heap.size();
}

// Time: O(n log n), Space: O(n)
```

Summary: Key Patterns

Binary Search Patterns

Pattern	Use Case	Example
Standard	Find exact match	Search in sorted array
Left/Right bound	Find first/last occurrence	Search Range
Search on answer	Optimization problems	Ship Capacity, Koko
Rotated array	Modified sorted array	Search Rotated

Heap Patterns

Pattern	Use Case	Example
Top K	K largest/smallest	Top K Frequent
Two heaps	Median tracking	Find Median
Merge K	Multiple sorted sources	Merge K Lists

Bit Manipulation

Pattern	Use Case	Example
XOR	Find unique element	Single Number
Bit counting	Count occurrences	Single Number II
Bit masks	Subset representation	Subsets

Greedy Patterns

Pattern	Use Case	Example
Local optimal	Interval scheduling	Jump Game
Sorting + sweep	Interval problems	Meeting Rooms
Two pointers	Optimization	Gas Station

Time Complexity Cheat Sheet

Algorithm	Time	Space
Binary Search	O(log n)	O(1)
Heap push/pop	O(log n)	O(n)
Trie insert/search	O(m)	O(n*m)
QuickSelect	O(n) avg	O(1)
Merge K Lists	O(n log k)	O(k)

This completes the DSA Solutions Guide!

Files created: - Part 1: Arrays & Strings - Part 2: Linked Lists, Stacks & Queues - Part 3: Trees & Graphs - Part 4: Dynamic Programming & Backtracking - Part 5: Binary Search, Heaps, Tries & More